AXI Interconnect User Guide

V1.0

Efinix

2020-12-17

修订记录

日期	修订版本	描述	编写
2020-12-17	V1.0	初版	李 峰（Jeffery Li）

目录

1 参数描述 1
2 接口描述 2
3 AXI4 协议简述 9
3.1 AXI 握手机制 9
3.2 AXI 突发读传输 10
3.3 AXI 突发写传输 11

图目录

图 1 VALID 在 READY 之前的握手.....10
图 2 READY 在 VALID 之前的握手.....10
图 3 VALID 和 READY 同时的握手.....10
图 4 突发读传输.....11
图 5 突发写传输.....11

表目录

表 1 参数定义.....1
表 2 接口信号定义.....2

1 参数描述

表 1 参数定义

<table>
<tr><th>Name</th><th>Description</th></tr>
<tr><td>S_COUNT</td><td>Slave 总线数。</td></tr>
<tr><td>M_COUNT</td><td>Master 总线数。需要注意的是 Master 集可以访问的最大地址宽度为 ADDR_WIDTH 位。当设置多个 Master 时，假设 ADDR_WIDTH 配置为 32 位，Master 集是根据 M_COUNT 来均匀分配这 32 位地址区间的。当 Slave 需要访问某一 Master 总线时，只需要访问该 Master 总线对应的地址区间即可。Master 集的地址区间分配如下：
<table>
<tr><th>M_COUNT</th><th>Port</th><th>Address Region</th></tr>
<tr><td>1</td><td>Master_0</td><td>0xFFFF_FFFF - 0x0000_0000</td></tr>
<tr><td rowspan="2">2</td><td>Master_0</td><td>0x7FFF_FFFF - 0x0000_0000</td></tr>
<tr><td>Master_1</td><td>0xFFFF_FFFF - 0x8000_0000</td></tr>
<tr><td rowspan="4">3</td><td>Master_0</td><td>0x3FFF_FFFF - 0x0000_0000</td></tr>
<tr><td>Master_1</td><td>0x7FFF_FFFF - 0x4000_0000</td></tr>
<tr><td>Master_2</td><td>0xBFFF_FFFF - 0x8000_0000</td></tr>
<tr><td>Null</td><td>Null</td></tr>
<tr><td rowspan="4">4</td><td>Master_0</td><td>0x3FFF_FFFF - 0x0000_0000</td></tr>
<tr><td>Master_1</td><td>0x7FFF_FFFF - 0x4000_0000</td></tr>
<tr><td>Master_2</td><td>0xBFFF_FFFF - 0x8000_0000</td></tr>
<tr><td>Master_3</td><td>0xFFFF_FFFF - 0xC000_0000</td></tr>
<tr><td colspan="3">...</td></tr>
</table>
</td></tr>
<tr><td>DATA_WIDTH</td><td>总线数据位宽。</td></tr>
<tr><td>ADDR_WIDTH</td><td>总线地址位宽。</td></tr>
<tr><td>ID_WIDTH</td><td>总线 ID 位宽。未使用时置 1。</td></tr>
<tr><td>AWUSER_ENABLE</td><td>写地址用户自定义信号使能。未使用时置 0。</td></tr>
<tr><td>AWUSER_WIDTH</td><td>写地址用户自定义信号位宽。未使用时置 1。</td></tr>
<tr><td>WUSER_ENABLE</td><td>写数据用户自定义信号使能。未使用时置 0。</td></tr>
<tr><td>WUSER_WIDTH</td><td>写数据用户自定义信号位宽。未使用时置 1。</td></tr>
<tr><td>BUSER_ENABLE</td><td>写响应用户自定义信号使能。未使用时置 0。</td></tr>
<tr><td>BUSER_WIDTH</td><td>写响应用户自定义信号位宽。未使用时置 1。</td></tr>
<tr><td>ARUSER_ENABLE</td><td>读地址用户自定义信号使能。未使用时置 0。</td></tr>
<tr><td>ARUSER_WIDTH</td><td>读地址用户自定义信号位宽。未使用时置 1。</td></tr>
<tr><td>RUSER_ENABLE</td><td>读数据用户自定义信号使能。未使用时置 0。</td></tr>
<tr><td>RUSER_WIDTH</td><td>读数据用户自定义信号位宽。未使用时置 1。</td></tr>
<tr><td>FORWARD_ID</td><td>传送 ID 使能。是否向 Master 总线传送来自于 Slave 总线的 ID 信号。未使用时置 0。</td></tr>
<tr><td>M_REGIONS</td><td>默认为 1。常规应用维持默认值，不配置。</td></tr>
<tr><td>M_BASE_ADDR</td><td>Master 总线基准地址。默认为 0。常规应用维持默认值，不配置。</td></tr>
<tr><td>M_ADDR_WIDTH</td><td>Master 总线的地址区间。默认值为</td></tr>
</table>

	{M_COUNT{{M_REGIONS{ADDR_WIDTH-clogb2(M_COUNT)}}}}。常规应用维持默认值，不配置。
M_CONNECT_READ	Master 总线对 Slave 总线读通道使能。每一 Slave 总线的读通道对应一位使能控制。置 1 使能，置 0 无效。 默认值为{M_COUNT{{S_COUNT{1'b1}}}}。常规应用维持默认值，不配置。
M_CONNECT_WRITE	Master 总线对 Slave 总线写通道使能。每一 Slave 总线的写通道对应一位使能控制。置 1 使能，置 0 无效。 默认值为{M_COUNT{{S_COUNT{1'b1}}}}。常规应用维持默认值，不配置。

2 接口描述

表 2 接口信号定义

<table>
<tr><th>Name</th><th>I/O</th><th>Description</th></tr>
<tr><td colspan="3">时钟与复位信号</td></tr>
<tr><td>clk</td><td>I</td><td>时钟</td></tr>
<tr><td>rst</td><td>I</td><td>复位。高有效。</td></tr>
<tr><td colspan="3">AXI4 Slave 接口</td></tr>
<tr><td>s_axi_awid
[S_COUNT*ID_WIDTH-1:0]</td><td>I</td><td>写地址 ID。</td></tr>
<tr><td>s_axi_awaddr
[S_COUNT*ADDR_WIDTH-1:0]</td><td>I</td><td>写地址。</td></tr>
<tr><td>s_axi_awlen
[S_COUNT*8-1:0]</td><td>I</td><td>写地址 Burst 长度。表示一次 Burst 传输的周期数。Burst_Length = Axlen[7:0] + 1。</td></tr>
<tr><td>s_axi_awsize
[S_COUNT*3-1:0]</td><td>I</td><td>写地址 Burst 大小。表示每一周期传输最大的字节数。
Byte_Length = $2^{Axsize[2:0]}$。</td></tr>
<tr><td>s_axi_awburst
[S_COUNT*2-1:0]</td><td>I</td><td>写地址 Burst 类型。
<table>
<tr><th>AxBURST[1:0]</th><th>Burst Type</th><th>Description</th></tr>
<tr><td>00b</td><td>FIXED</td><td>FIXED 类型是指地址是固定的，每一次传输的地址都不变，重复对同一个地址进行存取。典型应用：FIFO。</td></tr>
<tr><td>01b</td><td>INCR</td><td>INCR 类型是指每一次读写的地址都比上一次增加一个固定值。</td></tr>
<tr><td>10b</td><td>WRAP</td><td>WRAP 类型与 INCR 类型类似，每次读写地址增加一个固定</td></tr>
</table></td></tr>
</table>

<table>
<tr><td></td><td></td><td>
<table>
<tr><td></td><td></td><td>值。不同的是 WRAP 类型在地址增加到上限后，地址需要跳转到低地址。WRAP 还有以下限制：1）起始地址必须与 Axsize 大小对齐；2）Burst 长度必须是 2/4/8/16。</td></tr>
<tr><td>11b</td><td>Reservd</td><td></td></tr>
</table>
</td></tr>
<tr><td>s_axi_awlock
[S_COUNT-1:0]</td><td>I</td><td>写地址锁定类型。AXI4 不支持锁定事务。(AXI3 必须支持锁定事务)</td></tr>
<tr><td>s_axi_awcache
[S_COUNT*4-1:0]</td><td>I</td><td>写地址 Memory 类型。</td></tr>
<tr><td>s_axi_awprot
[S_COUNT*3-1:0]</td><td>I</td><td>写地址保护类型。</td></tr>
<tr><td>s_axi_awqos
[S_COUNT*4-1:0]</td><td>I</td><td>写地址 QoS 标识。AXI4 信号集被扩展为支持两个 4 位 QoS 标识符，分别对应 AWQOS 和 ARQOS。</td></tr>
<tr><td>s_axi_awuser
[S_COUNT*AWUSER_WIDTH-1:0]</td><td>I</td><td>写地址用户自定义信号。</td></tr>
<tr><td>s_axi_awvalid
[S_COUNT-1:0]</td><td>I</td><td>写地址有效信号。</td></tr>
<tr><td>s_axi_awready
[S_COUNT-1:0]</td><td>O</td><td>写地址准备好信号。</td></tr>
<tr><td>s_axi_wdata
[S_COUNT*DATA_WIDTH-1:0]</td><td>I</td><td>写数据。</td></tr>
<tr><td>s_axi_wstrb
[S_COUNT*STRB_WIDTH-1:0]</td><td>I</td><td>写数据闸门。这个信号表明哪些字节为有效的数据。每一位对应一字节数据。高有效。</td></tr>
<tr><td>s_axi_wlast
[S_COUNT-1:0]</td><td>I</td><td>写数据结束。表示一次 Burst 传输的最后一个传输周期。</td></tr>
<tr><td>s_axi_wuser
[S_COUNT*WUSER_WIDTH-1:0]</td><td>I</td><td>写数据用户自定义信号。</td></tr>
<tr><td>s_axi_wvalid
[S_COUNT-1:0]</td><td>I</td><td>写数据有效信号。</td></tr>
<tr><td>s_axi_wready
[S_COUNT-1:0]</td><td>O</td><td>写数据准备好信号。</td></tr>
<tr><td>s_axi_bid
[S_COUNT*ID_WIDTH-1:0]</td><td>O</td><td>写响应 ID。该数值必须与写地址 ID 匹配。</td></tr>
<tr><td>s_axi_bresp
[S_COUNT*2-1:0]</td><td>O</td><td>写响应。
<table>
<tr><th>BRESP[1:0]</th><th>Response</th><th>Description</th></tr>
<tr><td>00b</td><td>OKAY</td><td>Normal 访问成功。</td></tr>
</table>
</td></tr>
</table>

<table>
<tr><td></td><td></td><td>
<table>
<tr><td></td><td></td><td></td></tr>
<tr><td>01b</td><td>EXOKAY</td><td>Exclusive 访问成功。</td></tr>
<tr><td>10b</td><td>SLVERR</td><td>Slave 错误。指示访问错误。</td></tr>
<tr><td>11b</td><td>DECERR</td><td>Decode 错误。通常用来指示没有访问对应的访问地址。</td></tr>
</table>
</td></tr>
<tr><td>s_axi_buser
[S_COUNT*BUSER_WIDTH-1:0]</td><td>O</td><td>写响应用户自定义信号。</td></tr>
<tr><td>s_axi_bvalid
[S_COUNT-1:0]</td><td>O</td><td>写响应有效信号。</td></tr>
<tr><td>s_axi_bready
[S_COUNT-1:0]</td><td>I</td><td>写响应准备好信号。</td></tr>
<tr><td>s_axi_arid
[S_COUNT*ID_WIDTH-1:0]</td><td>I</td><td>读地址 ID。</td></tr>
<tr><td>s_axi_araddr
[S_COUNT*ADDR_WIDTH-1:0]</td><td>I</td><td>读地址。</td></tr>
<tr><td>s_axi_arlen
[S_COUNT*8-1:0]</td><td>I</td><td>读地址 Burst 长度。表示一次 Burst 传输的周期数。Burst_Length = Axlen[7:0] + 1。</td></tr>
<tr><td>s_axi_arsize
[S_COUNT*3-1:0]</td><td>I</td><td>读地址 Burst 大小。表示每一周期传输最大的字节数。
Byte_Length = $2^{Axsize[2:0]}$。</td></tr>
<tr><td>s_axi_arburst
[S_COUNT*2-1:0]</td><td>I</td><td>读地址 Burst 类型。
<table>
<tr><th>AxBURST[1:0]</th><th>Burst Type</th><th>Description</th></tr>
<tr><td>00b</td><td>FIXED</td><td>FIXED 类型是指地址是固定的，每一次传输的地址都不变，重复对同一个地址进行存取。典型应用：FIFO。</td></tr>
<tr><td>01b</td><td>INCR</td><td>INCR 类型是指每一次读写的地址都比上一次增加一个固定值。</td></tr>
<tr><td>10b</td><td>WRAP</td><td>WRAP 类型与 INCR 类型类似，每次读写地址增加一个固定值。不同的是 WRAP 类型在地址增加到上限后，地址需要跳转到低地址。WRAP 还有以下限制：1）起</td></tr>
</table>
</td></tr>
</table>

<table>
<tr><td></td><td></td><td>
<table>
<tr><td></td><td></td><td>始地址必须与 Axsize 大小对齐；2）Burst 长度必须是 2/4/8/16。</td></tr>
<tr><td>11b</td><td>Reservd</td><td></td></tr>
</table>
</td></tr>
<tr><td>s_axi_arlock
[S_COUNT-1:0]</td><td>I</td><td>读地址锁定类型。AXI4 不支持锁定事务。(AXI3 必须支持锁定事务)</td></tr>
<tr><td>s_axi_arcache
[S_COUNT*4-1:0]</td><td>I</td><td>读地址 Memory 类型。</td></tr>
<tr><td>s_axi_arprot
[S_COUNT*3-1:0]</td><td>I</td><td>读地址保护类型。</td></tr>
<tr><td>s_axi_arqos
[S_COUNT*4-1:0]</td><td>I</td><td>读地址 QoS 标识。AXI4 信号集被扩展为支持两个 4 位 QoS 标识符，分别对应 AWQOS 和 ARQOS。</td></tr>
<tr><td>s_axi_aruser
[S_COUNT*ARUSER_WIDTH-1:0]</td><td>I</td><td>读地址用户自定义信号。</td></tr>
<tr><td>s_axi_arvalid
[S_COUNT-1:0]</td><td>I</td><td>读地址有效信号。</td></tr>
<tr><td>s_axi_arready
[S_COUNT-1:0]</td><td>O</td><td>读地址准备好信号。</td></tr>
<tr><td>s_axi_rid
[S_COUNT*ID_WIDTH-1:0]</td><td>O</td><td>读数据 ID。该数值必须与读地址 ID 匹配。</td></tr>
<tr><td>s_axi_rdata
[S_COUNT*DATA_WIDTH-1:0]</td><td>O</td><td>读数据。</td></tr>
<tr><td>s_axi_rresp
[S_COUNT*2-1:0]</td><td>O</td><td>读响应。
<table>
<tr><th>RRESP[1:0]</th><th>Response</th><th>Description</th></tr>
<tr><td>00b</td><td>OKAY</td><td>Normal 访问成功。</td></tr>
<tr><td>01b</td><td>EXOKAY</td><td>Exclusive 访问成功。</td></tr>
<tr><td>10b</td><td>SLVERR</td><td>Slave 错误。指示访问错误。</td></tr>
<tr><td>11b</td><td>DECERR</td><td>Decode 错误。通常用来指示没有访问对应的访问地址。</td></tr>
</table>
</td></tr>
<tr><td>s_axi_rlast
[S_COUNT-1:0]</td><td>O</td><td>读数据结束。表示一次 Burst 传输的最后一个传输周期。</td></tr>
<tr><td>s_axi_ruser
[S_COUNT*RUSER_WIDTH-1:0]</td><td>O</td><td>读数据用户自定义信号。</td></tr>
<tr><td>s_axi_rvalid
[S_COUNT-1:0]</td><td>O</td><td>读数据有效信号。</td></tr>
<tr><td>s_axi_rready
[S_COUNT-1:0]</td><td>I</td><td>读数据准备好信号。</td></tr>
</table>

<table>
<tr><th colspan="3">AXI4 Master 接口</th></tr>
<tr><td>m_axi_awid
[M_COUNT*ID_WIDTH-1:0]</td><td>O</td><td>写地址 ID。</td></tr>
<tr><td>m_axi_awaddr
[M_COUNT*ADDR_WIDTH-1:0]</td><td>O</td><td>写地址。</td></tr>
<tr><td>m_axi_awlen
[M_COUNT*8-1:0]</td><td>O</td><td>写地址 Burst 长度。表示一次 Burst 传输的周期数。Burst_Length = Axlen[7:0] + 1。</td></tr>
<tr><td>m_axi_awsize
[M_COUNT*3-1:0]</td><td>O</td><td>写地址 Burst 大小。表示每一周期传输最大的字节数。
Byte_Length = $2^{Axsize[2:0]}$。</td></tr>
<tr><td>m_axi_awburst
[M_COUNT*2-1:0]</td><td>O</td><td>写地址 Burst 类型。
<table>
<tr><th>AxBURST[1:0]</th><th>Burst Type</th><th>Description</th></tr>
<tr><td>00b</td><td>FIXED</td><td>FIXED 类型是指地址是固定的，每一次传输的地址都不变，重复对同一个地址进行存取。典型应用：FIFO。</td></tr>
<tr><td>01b</td><td>INCR</td><td>INCR 类型是指每一次读写的地址都比上一次增加一个固定值。</td></tr>
<tr><td>10b</td><td>WRAP</td><td>WRAP 类型与 INCR 类型类似，每次读写地址增加一个固定值。不同的是 WRAP 类型在地址增加到上限后，地址需要跳转到低地址。WRAP 还有以下限制：1）起始地址必须与 Axsize 大小对齐；2）Burst 长度必须是 2/4/8/16。</td></tr>
<tr><td>11b</td><td>Reservd</td><td></td></tr>
</table></td></tr>
<tr><td>m_axi_awlock
[M_COUNT-1:0]</td><td>O</td><td>写地址锁定类型。AXI4 不支持锁定事务。(AXI3 必须支持锁定事务)</td></tr>
<tr><td>m_axi_awcache
[M_COUNT*4-1:0]</td><td>O</td><td>写地址 Memory 类型。</td></tr>
<tr><td>m_axi_awprot
[M_COUNT*3-1:0]</td><td>O</td><td>写地址保护类型。</td></tr>
<tr><td>m_axi_awqos</td><td>O</td><td>写地址 QoS 标识。AXI4 信号集被扩展为支持两个 4</td></tr>
</table>

<table>
<tr><td>[M_COUNT*4-1:0]</td><td></td><td>位 QoS 标识符，分别对应 AWQOS 和 ARQOS。</td></tr>
<tr><td>m_axi_awregion
[M_COUNT*4-1:0]</td><td>O</td><td>写地址 Region 标识。AXI4 信号集被扩展为支持两个 4 位 Region 标识符，分别对应 AWREGION 和 ARREGION。</td></tr>
<tr><td>m_axi_awuser
[M_COUNT*AWUSER_WIDTH-1:0]</td><td>O</td><td>写地址用户自定义信号。</td></tr>
<tr><td>m_axi_awvalid
[M_COUNT-1:0]</td><td>O</td><td>写地址有效信号。</td></tr>
<tr><td>m_axi_awready
[M_COUNT-1:0]</td><td>I</td><td>写地址准备好信号。</td></tr>
<tr><td>m_axi_wdata
[M_COUNT*DATA_WIDTH-1:0]</td><td>O</td><td>写数据。</td></tr>
<tr><td>m_axi_wstrb
[M_COUNT*STRB_WIDTH-1:0]</td><td>O</td><td>写数据闸门。这个信号表明哪些字节为有效的数据。每一位对应一字节数据。高有效。</td></tr>
<tr><td>m_axi_wlast
[M_COUNT-1:0]</td><td>O</td><td>写数据结束。表示一次 Burst 传输的最后一个传输周期。</td></tr>
<tr><td>m_axi_wuser
[M_COUNT*WUSER_WIDTH-1:0]</td><td>O</td><td>写数据用户自定义信号。</td></tr>
<tr><td>m_axi_wvalid
[M_COUNT-1:0]</td><td>O</td><td>写数据有效信号。</td></tr>
<tr><td>m_axi_wready
[M_COUNT-1:0]</td><td>I</td><td>写数据准备好信号。</td></tr>
<tr><td>m_axi_bid
[M_COUNT*ID_WIDTH-1:0]</td><td>I</td><td>写响应 ID。该数值必须与写地址 ID 匹配。</td></tr>
<tr><td>m_axi_bresp
[M_COUNT*2-1:0]</td><td>I</td><td>写响应。
<table>
<tr><th>BRESP[1:0]</th><th>Response</th><th>Description</th></tr>
<tr><td>00b</td><td>OKAY</td><td>Normal 访问成功。</td></tr>
<tr><td>01b</td><td>EXOKAY</td><td>Exclusive 访问成功。</td></tr>
<tr><td>10b</td><td>SLVERR</td><td>Slave 错误。指示访问错误。</td></tr>
<tr><td>11b</td><td>DECERR</td><td>Decode 错误。通常用来指示没有访问对应的访问地址。</td></tr>
</table></td></tr>
<tr><td>m_axi_buser
[M_COUNT*BUSER_WIDTH-1:0]</td><td>I</td><td>写响应用户自定义信号。</td></tr>
<tr><td>m_axi_bvalid
[M_COUNT-1:0]</td><td>I</td><td>写响应有效信号。</td></tr>
</table>

<table>
<tr><td>m_axi_bready
[M_COUNT-1:0]</td><td>O</td><td>写响应准备好信号。</td></tr>
<tr><td>m_axi_arid
[M_COUNT*ID_WIDTH-1:0]</td><td>O</td><td>读地址 ID。</td></tr>
<tr><td>m_axi_araddr
[M_COUNT*ADDR_WIDTH-1:0]</td><td>O</td><td>读地址。</td></tr>
<tr><td>m_axi_arlen
[M_COUNT*8-1:0]</td><td>O</td><td>读地址 Burst 长度。表示一次 Burst 传输的周期数。
Burst_Length = Axlen[7:0] + 1。</td></tr>
<tr><td>m_axi_arsize
[M_COUNT*3-1:0]</td><td>O</td><td>读地址 Burst 大小。表示每一周期传输最大的字节数。
Byte_Length = $2^{Axsize[2:0]}$。</td></tr>
<tr><td>m_axi_arburst
[M_COUNT*2-1:0]</td><td>O</td><td>读地址 Burst 类型。
<table>
<tr><th>AxBURST[1:0]</th><th>Burst Type</th><th>Description</th></tr>
<tr><td>00b</td><td>FIXED</td><td>FIXED 类型是指地址是固定的，每一次传输的地址都不变，重复对同一个地址进行存取。典型应用：FIFO。</td></tr>
<tr><td>01b</td><td>INCR</td><td>INCR 类型是指每一次读写的地址都比上一次增加一个固定值。</td></tr>
<tr><td>10b</td><td>WRAP</td><td>WRAP 类型与 INCR 类型类似，每次读写地址增加一个固定值。不同的是 WRAP 类型在地址增加到上限后，地址需要跳转到低地址。WRAP 还有以下限制：1）起始地址必须与 Axsize 大小对齐；2）Burst 长度必须是 2/4/8/16。</td></tr>
<tr><td>11b</td><td>Reservd</td><td></td></tr>
</table></td></tr>
<tr><td>m_axi_arlock
[M_COUNT-1:0]</td><td>O</td><td>读地址锁定类型。AXI4 不支持锁定事务。(AXI3 必须支持锁定事务)</td></tr>
<tr><td>m_axi_arcache
[M_COUNT*4-1:0]</td><td>O</td><td>读地址 Memory 类型。</td></tr>
<tr><td>m_axi_arprot
[M_COUNT*3-1:0]</td><td>O</td><td>读地址保护类型。</td></tr>
</table>

<table>
<tr><td>m_axi_arqos
[M_COUNT*4-1:0]</td><td>O</td><td>读地址 QoS 标识。AXI4 信号集被扩展为支持两个 4 位 QoS 标识符，分别对应 AWQOS 和 ARQOS。</td></tr>
<tr><td>m_axi_arregion
[M_COUNT*4-1:0]</td><td>O</td><td>读地址 Region 标识。AXI4 信号集被扩展为支持两个 4 位 Region 标识符，分别对应 AWREGION 和 ARREGION。</td></tr>
<tr><td>m_axi_aruser
[M_COUNT*ARUSER_WIDTH-1:0]</td><td>O</td><td>读地址用户自定义信号。</td></tr>
<tr><td>m_axi_arvalid
[M_COUNT-1:0]</td><td>O</td><td>读地址有效信号。</td></tr>
<tr><td>m_axi_arready
[M_COUNT-1:0]</td><td>I</td><td>读地址准备好信号。</td></tr>
<tr><td>m_axi_rid
[M_COUNT*ID_WIDTH-1:0]</td><td>I</td><td>读数据 ID。该数值必须与读地址 ID 匹配。</td></tr>
<tr><td>m_axi_rdata
[M_COUNT*DATA_WIDTH-1:0]</td><td>I</td><td>读数据。</td></tr>
<tr><td>m_axi_rresp
[M_COUNT*2-1:0]</td><td>I</td><td>读响应。
<table>
<tr><th>RRESP[1:0]</th><th>Response</th><th>Description</th></tr>
<tr><td>00b</td><td>OKAY</td><td>Normal 访问成功。</td></tr>
<tr><td>01b</td><td>EXOKAY</td><td>Exclusive 访问成功。</td></tr>
<tr><td>10b</td><td>SLVERR</td><td>Slave 错误。指示访问错误。</td></tr>
<tr><td>11b</td><td>DECERR</td><td>Decode 错误。通常用来指示没有访问对应的访问地址。</td></tr>
</table></td></tr>
<tr><td>m_axi_rlast
[M_COUNT-1:0]</td><td>I</td><td>读数据结束。表示一次 Burst 传输的最后一个传输周期。</td></tr>
<tr><td>m_axi_ruser
[M_COUNT*RUSER_WIDTH-1:0]</td><td>I</td><td>读数据用户自定义信号。</td></tr>
<tr><td>m_axi_rvalid
[M_COUNT-1:0]</td><td>I</td><td>读数据有效信号。</td></tr>
<tr><td>m_axi_rready
[M_COUNT-1:0]</td><td>O</td><td>读数据准备好信号。</td></tr>
</table>

3 AXI4 协议简述

3.1 AXI 握手机制

AXI4 所采用的是一种 READY，VALID 握手通信机制，简单来说主从双方进行数据通信前，有一个握手的过程。传输源产生 VLAID 信号来指明何时数据或控制信息有效。

而目地源产生 READY 信号来指明已经准备好接受数据或控制信息。传输发生在 VALID 和 READY 信号同时为高的时候。如下图中的实例：

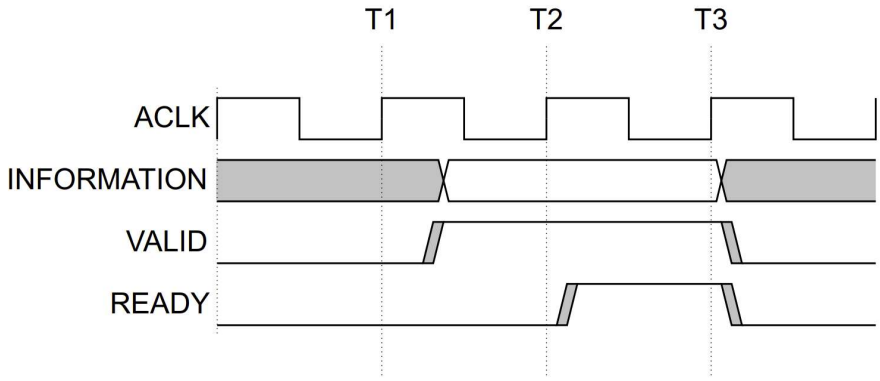

图 1 VALID 在 READY 之前的握手

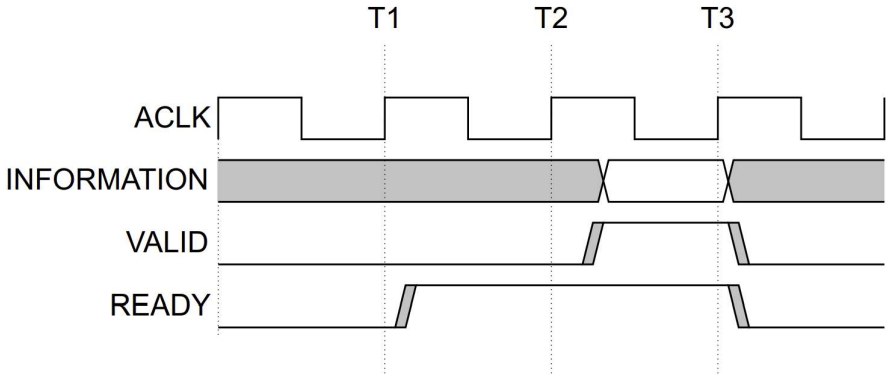

图 2 READY 在 VALID 之前的握手

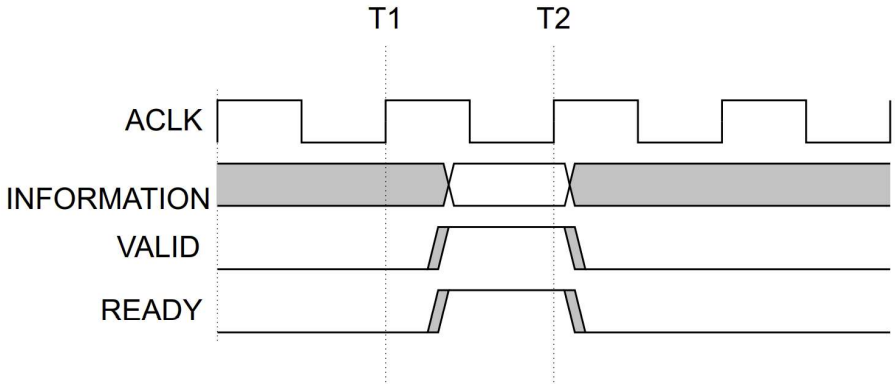

图 3 VALID 和 READY 同时的握手

3.2 AXI 突发读传输

当地址出现在地址总线后，传输的数据将出现在读数据通道上。设备保持 VALID 为低直到读数据有效。为了表明一次突发式读写的完成，设备用 RLAST 信号来表示最后一个被传输的数据。

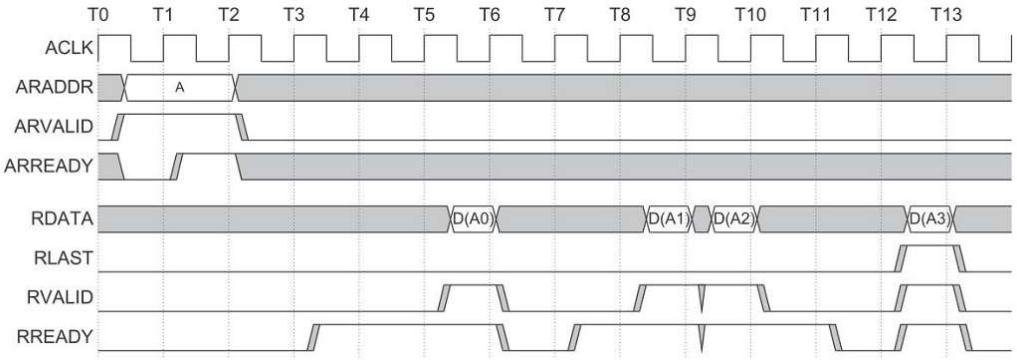

图 4 突发读传输

3.3 AXI 突发写传输

主机发送地址和控制信息到写地址通道中，然后主机发送每一个写数据到写数据通道中。当主机发送最后一个数据时，WLAST 信号就变为高。当设备接收完所有数据之后他将一个写响应发送回主机来表明写事务完成。

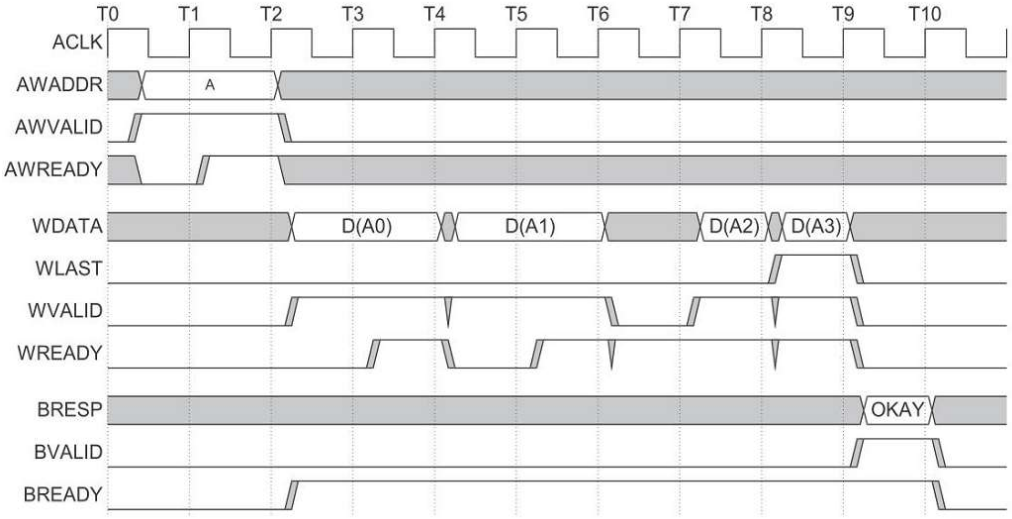

图 5 突发写传输